



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Batch: B2 Roll No.: 16010124107

Experiment No. 10

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of the Staff In-charge with date

Title: Study, Implementation, and Analysis of the Longest Common Subsequence Algorithm.

Objective: To compute longest common subsequence for the given two strings.

CO to be achieved:

CO 2	Analyze and solve problems for divide and conquer strategy, greedy method, dynamic programming approach and backtracking and branch & bound policies.
CO 3	Analyze and solve problems for different string matching algorithms.

Books/ Journals/ Websites referred:

1. Ellis horowitz, Sarataj Sahni, S.Rajsekaran," Fundamentals of computer algorithm", University Press
2. T.H.Cormen ,C.E.Leiserson,R.L.Rivest and C.Stein," Introduction to algorithms",2nd Edition ,MIT press/McGraw Hill,2001
3. <http://www.math.utah.edu/~alfeld/queens/queens>.

Pre Lab/ Prior Concepts:

Data structures, Concepts of algorithm analysis

Historical Profile:

Given 2 sequences, $X = x_1, \dots, x_m$ and $Y = y_1, \dots, y_n$, find a subsequence common to both whose length is longest. A subsequence doesn't have to be consecutive, but it has to be in order.

New Concepts to be learned:

String matching algorithm, Dynamic programming approach for LCS, Applications of LCS.



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Recursive Formulation:

Define $c[i, j]$ = length of LCS of X_i and Y_j .

Final answer will be computed with $c[m, n]$.

```
c[i, j] = 0
if i=0 or j=0.
c[i, j] = c[i - 1, j - 1] + 1
if i, j > 0 and  $x_i = y_j$ 

c[i, j] = max(c[i - 1, j], c[i, j - 1])
if i, j > 0 and  $x_i \neq y_j$ 
```

Algorithm: Longest Common Subsequence

Compute length of optimal solution-

LCS-LENGTH (X, Y, m, n)

for $i \leftarrow 1$ **to** m

do $c[i, 0] \leftarrow 0$

for $j \leftarrow 0$ **to** n

do $c[0, j] \leftarrow 0$

for $i \leftarrow 1$ **to** m

do for $j \leftarrow 1$ **to** n

do if $x_i = y_j$

then $c[i, j] \leftarrow c[i - 1, j - 1] + 1$

$b[i, j] \leftarrow \approx$

else if $c[i - 1, j] \geq c[i, j - 1]$

then $c[i, j] \leftarrow c[i - 1, j]$

$b[i, j] \leftarrow \uparrow$

else $c[i, j] \leftarrow c[i, j - 1]$

$b[i, j] \leftarrow \leftarrow$

return c and b

Print the solution-

PRINT-LCS(b, X, i, j)

if $i = 0$ or $j = 0$

then return

if $b[i, j] = \approx$

then PRINT-LCS($b, X, i - 1, j - 1$)

 print x_i

elseif $b[i, j] = \uparrow$

then PRINT-LCS($b, X, i - 1, j$)

else PRINT-LCS($b, X, i, j - 1$)



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Initial call is PRINT-LCS(b, X, m, n).

$b[i, j]$ points to table entry whose subproblem we used in solving LCS of X_i and Y_j .

When $b[i, j] = \approx$, we have extended LCS by one character. So longest common subsequence = entries with $\approx$ in them.

Example: LCS computation

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define yes cout << "YES" << endl;
#define no cout << "NO" << endl;
#define input for(int &i:v) cin >> i;
#define vall v.begin(),v.end()

void print(const vector<int>& v) {
    for (int i : v) cout << i << " ";
    cout << endl;
}

void solve() {
    string s1, s2;
    cin >> s1 >> s2;

    int n = s1.size(), m = s2.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            if (s1[i-1] == s2[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);

    cout << "LCS Length: " << dp[n][m] << "\n";

    string lcs;
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (s1[i-1] == s2[j-1]) {
```



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

```
        lcs += s1[i-1];
        i--; j--;
    } else if (dp[i-1][j] > dp[i][j-1]) {
        i--;
    } else {
        j--;
    }
}

reverse(lcs.begin(), lcs.end());
cout << "LCS: " << lcs << "\n";
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    solve();
}
```

Analysis of LCS computation

The Longest Common Subsequence problem is solved using dynamic programming by breaking it into smaller overlapping subproblems. The solution builds a table that stores the length of the LCS for different prefixes of the two strings. Each cell represents the result for a pair of substring lengths.

If the characters at the current positions are equal, the value is obtained by adding one to the result of the previous smaller subproblem. If the characters are not equal, the value is taken as the maximum of the two possible previous subproblems. This ensures that the optimal solution is built step by step.

The time complexity of this approach is $O(m \times n)$, where m and n are the lengths of the two strings, because every cell in the table is computed once. The space complexity is also $O(m \times n)$ due to the storage of the table. This can be optimized to $O(\min(m, n))$ by storing only the necessary rows.

Output:



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

```
PS C:\Users\syen  
er 2\Analysis  
gestCommonSubse  
abuela  
buelea  
LCS Length: 5  
LCS: buela
```

Algorithm:

1. Start
2. Read two strings s1 and s2
3. Find length of both strings n and m
4. Create a 2D array dp of size $(n+1) \times (m+1)$
Initialize all values as 0
5. For i from 1 to n
 - a. For j from 1 to m
 - b. If characters s1[i-1] and s2[j-1] are equal
Set $dp[i][j] = dp[i-1][j-1] + 1$
 - c. Else
Set $dp[i][j] = \text{maximum of } dp[i-1][j] \text{ and } dp[i][j-1]$
 - d. End inner loop
6. End outer loop
7. The value $dp[n][m]$ gives length of LCS
8. To find actual LCS string
9. Set $i = n$ and $j = m$
10. Initialize empty string lcs
11. While $i > 0$ and $j > 0$
 - a. If s1[i-1] equals s2[j-1]
Add character to lcs
Move diagonally i-- and j--
 - b. Else if $dp[i-1][j]$ is greater
Move up i--
 - c. Else
Move left j--
12. End while
13. Reverse the lcs string
14. Print LCS length and LCS string
15. Stop

CONCLUSION:

In conclusion, the dynamic programming approach guarantees an optimal solution by systematically storing intermediate results and avoiding repeated computation, making it efficient compared to a naive recursive solution.